

Natural Language Processing 2

Recurrent Neural Networks

Fatemeh Dehrouyeh



FANSHAWE

- 1 Recurrent Neural Networks
 - How Basic RNN Works?
 - Why LSTM was created?
 - How GRU is different from LSTM

Current Section

1 Recurrent Neural Networks

- How Basic RNN Works?
- Why LSTM was created?
- How GRU is different from LSTM

FFNN vs. RNN in NLP Classification

Feed-Forward Neural Network (FFNN)

- Uses one fixed-size vector for each document.
- Common input: TF-IDF vector or averaged word embedding vector.
- Does not directly model word order.
- Works well as a simple and fast baseline for text classification.

Document $\rightarrow$ Fixed-Size Vector $\rightarrow$ FFNN $\rightarrow$ Class

Recurrent Neural Network (RNN)

- Reads text as a sequence of words or tokens.
- Processes words step by step.
- Can use word order and previous context.
- Better when sentence structure changes the meaning.

word₁ $\rightarrow$ word₂ $\rightarrow$ $\dots$ $\rightarrow$ word_n $\rightarrow$ RNN $\rightarrow$ Class

Key Difference: FFNN sees a document-level representation, while RNN sees the ordered sequence of words.

RNN vs. FFNN

FFNN classification: document is converted to one vector before the model.

RNN classification: document stays as a sequence, then the RNN creates the summary internally.

Dataset Example: Household Electric Power Consumption

Source: [Link](#)

```
# Read Household Power Consumption dataset

import pandas as pd

# 1. Load dataset
df = pd.read_csv("household_power_consumption.txt", sep=";", low_memory=False)

# 2. Show first rows
df.head()
```

	Date	Time	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering_2	Sub_metering_3
0	16/12/2006	17:24:00	4.216	0.418	234.840	18.400	0.000	1.000	17.0
1	16/12/2006	17:25:00	5.360	0.436	233.630	23.000	0.000	1.000	16.0
2	16/12/2006	17:26:00	5.374	0.498	233.290	23.000	0.000	2.000	17.0
3	16/12/2006	17:27:00	5.388	0.502	233.740	23.000	0.000	1.000	17.0
4	16/12/2006	17:28:00	3.666	0.528	235.680	15.800	0.000	1.000	17.0

Typical prediction task:

Past Power Usage → RNN/LSTM/GRU → Future Power Usage

Target example: `Global_active_power`, which represents the household's active power consumption.

Forecasting Setups with Lookback Window

Goal: predict future `Global_active_power` using the past L time steps.

1. Univariate forecasting

Use only past values of the target variable:

$$\begin{bmatrix} \text{GAP}_{t-3} \\ \text{GAP}_{t-2} \\ \text{GAP}_{t-1} \end{bmatrix} \rightarrow \text{GAP}_t$$

Here, $L = 3$, so the model uses the previous 3 time steps to predict the next value.

2. Multivariate forecasting

Use multiple features from the past L time steps:

$$\begin{bmatrix} \text{GAP}_{t-3} & \text{Voltage}_{t-3} & \text{Intensity}_{t-3} \\ \text{GAP}_{t-2} & \text{Voltage}_{t-2} & \text{Intensity}_{t-2} \\ \text{GAP}_{t-1} & \text{Voltage}_{t-1} & \text{Intensity}_{t-1} \end{bmatrix} \rightarrow \text{GAP}_t$$

Key Idea: For RNNs, each training input has shape $L \times F$, where L is the lookback window and F is the number of features.

One-Step vs. Multi-Step Forecasting

There are two common forecasting setups:

1. One-step forecasting

- The model uses the past L time steps to predict the next single value.
- This is the simplest setup and the output layer has one node.

$$[x_{t-L}, x_{t-L+1}, \dots, x_{t-1}] \rightarrow x_t$$

Example: Past 24 hours $\rightarrow$ predict the next hour.

2. Multi-step forecasting

- The model uses the past L time steps to predict multiple future values.
- The output layer has one node for each future time step.

$$[x_{t-L}, x_{t-L+1}, \dots, x_{t-1}] \rightarrow [x_t, x_{t+1}, \dots, x_{t+H-1}]$$

Example: Past 24 hours $\rightarrow$ predict the next H hours.

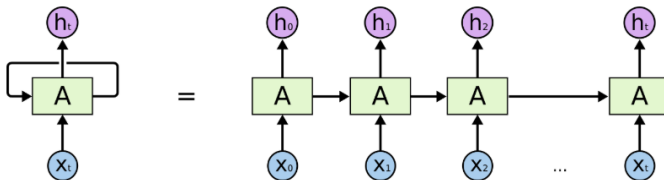
Where L is called the sequence length or lookback window, and each x_t can contain one feature or multiple features.

Current Subsection

- 1 Recurrent Neural Networks
 - How Basic RNN Works?
 - Why LSTM was created?
 - How GRU is different from LSTM

Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a type of neural network designed to process sequential data.



An unrolled recurrent neural network.

Image Ref: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Sliding Window for RNN Input

Main Idea (univariate one-step forecasting): RNNs learn from a sequence of past values to predict the next value. Let's assume:

$$x_t = \text{Global_active_power at time } t$$

Original Time Series



Sliding window with lookback length $L = 3$:

Input Sequence		Target
$[x_1, x_2, x_3]$	$\rightarrow$	x_4
$[x_2, x_3, x_4]$	$\rightarrow$	x_5
$[x_3, x_4, x_5]$	$\rightarrow$	x_6
$[x_4, x_5, x_6]$	$\rightarrow$	x_7
$[x_5, x_6, x_7]$	$\rightarrow$	x_8

Key Idea: Each training sample contains past values as input and the next value as the prediction target. The model learns from patterns across time, not from isolated rows.

Code Example—Creating RNN Input

```
# Sliding Window Example for Household Power Consumption Dataset

import pandas as pd
import numpy as np

# 1. Load dataset
df = pd.read_csv("household_power_consumption.txt", sep=";", low_memory=False)

# 2. Keep only the target column
df = df[["Global_active_power"]]

# 3. Convert values to numeric
df["Global_active_power"] = pd.to_numeric(df["Global_active_power"], errors="coerce")

# 4. Remove missing values
df = df.dropna()

# 5. Take a small sample for now
gap = df["Global_active_power"].values[:8]

print("Original Global_active_power time series:")
print(gap)

# 6. Create sliding window samples
L = 3

X = []
y = []

for i in range(len(gap) - L):
    X.append(gap[i:i+L])
    y.append(gap[i+L])

X = np.array(X)
y = np.array(y)
```

Code Example—Result

```
# 7. Show training samples
print("\nSliding window samples with L = 3:")
for i in range(len(X)):
    print(f"{X[i]} -> {y[i]}")

# 8. Show shapes
print("\nX shape:", X.shape)
print("y shape:", y.shape)
```

```
... Original Global_active_power time series:
[4.216 5.36  5.374 5.388 3.666 3.52  3.702 3.7  ]
```

```
Sliding window samples with L = 3:
```

```
[4.216 5.36  5.374] -> 5.388
[5.36  5.374 5.388] -> 3.666
[5.374 5.388 3.666] -> 3.52
[5.388 3.666 3.52 ] -> 3.702
[3.666 3.52  3.702] -> 3.7
```

```
X shape: (5, 3)
y shape: (5,)
```

How an RNN Processes a Sequence

Main Idea: An RNN reads the sequence step by step and updates a hidden state.

- At each time step, the RNN receives the current input x_t .
- It also uses the previous hidden state h_{t-1} .
- It produces a new hidden state h_t .
- The hidden state acts as memory of previous time steps.

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

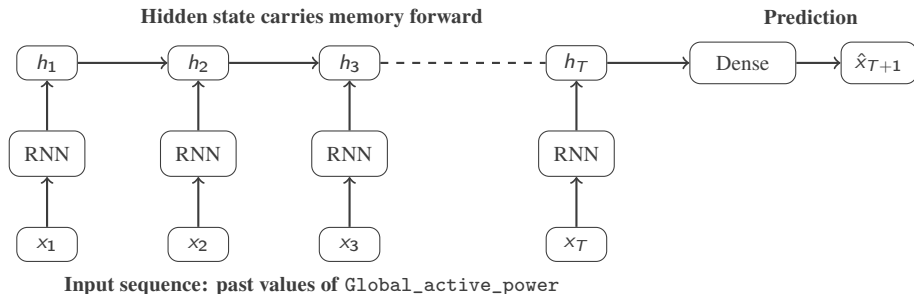
$$x_1 \rightarrow h_1 \rightarrow x_2 \rightarrow h_2 \rightarrow \cdots \rightarrow x_T \rightarrow h_T \rightarrow \hat{y}$$

For forecasting: The final hidden state can be used to predict the next value.

$$h_T \rightarrow \text{Dense Layer} \rightarrow \hat{y}_{T+1}$$

How an RNN Processes a Sequence

Main Idea: An RNN reads a sequence one step at a time. At each step, it updates a hidden state that carries information from the past.



$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

Key Idea: At time t , the RNN uses the current value x_t and the previous memory h_{t-1} to create a new memory h_t .

RNN Weights and Hidden State Size

RNN update equation:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

What each term means:

- x_t : input at time t , such as `Global_active_power`.
- h_{t-1} : hidden state from the previous time step.
- h_t : new hidden state after reading x_t .
- W_x : weights that connect the current input to the hidden state.
- W_h : weights that connect the previous hidden state to the new hidden state.
- b : bias term.
- f : activation function, commonly $\tanh$.

Important dimensions:

L = lookback length, F = number of features, H = number of hidden units

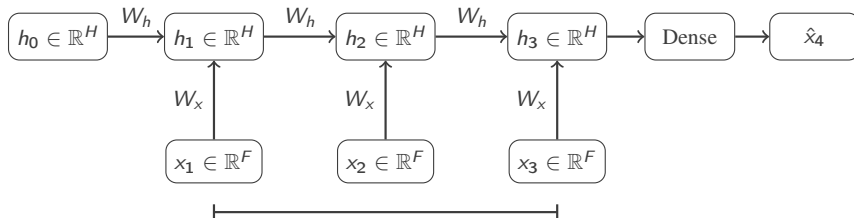
$$x_t \in \mathbb{R}^F, \quad h_t \in \mathbb{R}^H, \quad W_x \in \mathbb{R}^{H \times F}, \quad W_h \in \mathbb{R}^{H \times H}$$

RNN Dimensions and Weight Matrices

Example: Use $L = 3$ past time steps, $F = 1$ feature, and $H = 8$ hidden units.

$$x_t \in \mathbb{R}^F, \quad h_t \in \mathbb{R}^H, \quad W_x \in \mathbb{R}^{H \times F}, \quad W_h \in \mathbb{R}^{H \times H}$$

Each hidden state h_t has H values. Example: $H = 8$.



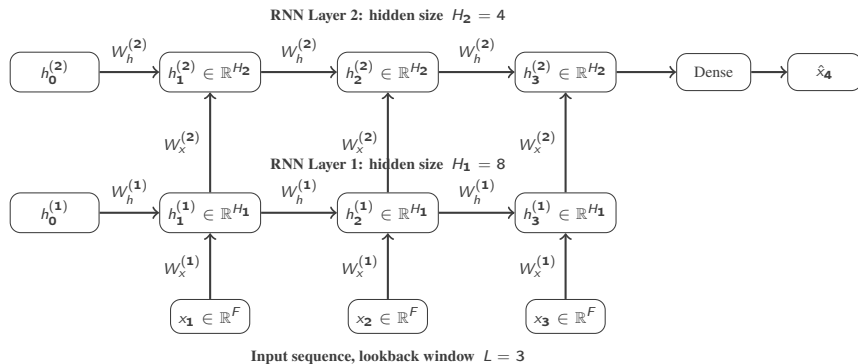
Each input x_t has F features. Example: $F = 1$ for only `Global_active_power`.

Key Idea: L controls how many time steps are read (how many times the RNN cell is repeated through time, selected by user as a data preparation choice), F controls the input size at each step, and H is the memory size chosen by the user.

Two-Layer RNN Dimensions

Example: Use $L = 3$ past time steps, $F = 1$ feature, $H_1 = 8$ hidden units in Layer 1, and $H_2 = 4$ hidden units in Layer 2.

$$x_t \in \mathbb{R}^F, \quad h_t^{(1)} \in \mathbb{R}^{H_1}, \quad h_t^{(2)} \in \mathbb{R}^{H_2}$$



Key Idea: The first RNN layer processes the original input sequence. The second RNN layer processes the sequence of hidden states produced by the first layer.

Code Example: Building a Basic RNN

Goal

Predict the next `Global_active_power` value.

Setting	Value
Lookback window	$L = 24$
First RNN layer	$H_1 = 64$ hidden units
Second RNN layer	$H_2 = 32$ hidden units
Output	One future GAP value

Past 24 GAP values $\rightarrow$ RNN(64) $\rightarrow$ RNN(32) $\rightarrow$ $\widehat{\text{Next GAP}}$

GAP = `Global_active_power`

Code Example: Building a Basic RNN

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# 1. Load dataset
df = pd.read_csv("household_power_consumption.txt", sep=";", low_memory=False)

# 2. Keep target column
df = df[["Global_active_power"]]

# 3. Convert to numeric and remove missing values
df["Global_active_power"] = pd.to_numeric(df["Global_active_power"], errors="coerce")
df = df.dropna()

# 4. Use a small part for now
data = df["Global_active_power"].values[:10000].reshape(-1, 1)

# 5. Scale data
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)

# 6. Choose lookback window L
L = 24

# 7. Create input sequences and targets
X = []
y = []

for i in range(len(data_scaled) - L):
    X.append(data_scaled[i:i+L])
    y.append(data_scaled[i+L])

X = np.array(X)
y = np.array(y)
```

Code Example: Building a Basic RNN—Continued

```
# 8. Show shapes
print("Lookback window L:", L)
print("X shape:", X.shape)
print("y shape:", y.shape)

# 9. Show one training sample
print("\nFirst input sequence:")
print(X[0].flatten())

print("\nFirst target:")
print(y[0])

... Lookback window L: 24
X shape: (9976, 24, 1)
y shape: (9976, 1)

First input sequence:
[0.52301691 0.67178153 0.67360208 0.67542263 0.45149545 0.43250975
 0.45617685 0.45591678 0.45175553 0.45097529 0.55318596 0.67854356
 0.65409623 0.65981795 0.50195059 0.41482445 0.4          0.42080624
 0.39947984 0.45955787 0.74122237 0.97685306 0.88842653 0.64759428]

First target:
[0.55656697]
```

```
# X shape = (samples, time steps, features)
# y shape = (samples, target values)
```

Code Example: Building a Basic RNN—Continued

```
# RNN model

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([
    SimpleRNN(64, activation="tanh", return_sequences=True, input_shape=(L, 1)),
    SimpleRNN(32, activation="tanh"),
    Dense(1)
])

model.compile(optimizer="adam", loss="mse", metrics=["mae"])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
simple_rnn_2 (SimpleRNN)	(None, 24, 64)	4,224
simple_rnn_3 (SimpleRNN)	(None, 32)	3,104
dense_1 (Dense)	(None, 1)	33

Total params: 7,361 (28.75 KB)
Trainable params: 7,361 (28.75 KB)
Non-trainable params: 0 (0.00 B)

Code Example: Building a Basic RNN—Continued

```
# Train, evaluate, and predict

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from tensorflow.keras.callbacks import EarlyStopping
import numpy as np

# 1. Train, test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, shuffle=False)

# 2. Early stopping
early_stop = EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True)

# 3. Train model
history = model.fit(X_train, y_train, validation_split=0.20, epochs=50, batch_size=32, callbacks=[early_stop], verbose=1)

# 4. Evaluate model
test_loss, test_mae = model.evaluate(X_test, y_test, verbose=0)
print("Test MSE:", test_loss)
print("Test MAE:", test_mae)

# 5. Predict
y_pred = model.predict(X_test)

# 6. Extra evaluation metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)

# 7. Compare actual vs predicted values
results = pd.DataFrame({"Actual": y_test.flatten(), "Predicted": y_pred.flatten()})
print(results.head(10))
```

Code Example: Building a Basic RNN—Result

```

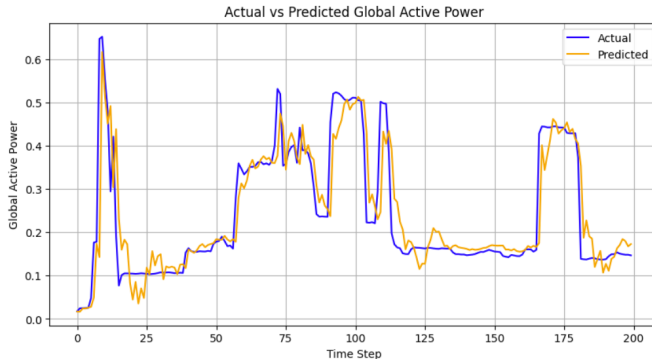
Epoch 1/50
200/200 ————— 6s 15ms/step - loss: 0.0096 - mae: 0.0563 - val_loss: 0.0021 - val_mae: 0.0236
Epoch 2/50
200/200 ————— 2s 11ms/step - loss: 0.0036 - mae: 0.0349 - val_loss: 0.0017 - val_mae: 0.0217
Epoch 3/50
200/200 ————— 3s 11ms/step - loss: 0.0032 - mae: 0.0324 - val_loss: 0.0016 - val_mae: 0.0202
Epoch 4/50
200/200 ————— 2s 10ms/step - loss: 0.0033 - mae: 0.0340 - val_loss: 0.0015 - val_mae: 0.0194
Epoch 5/50
200/200 ————— 3s 16ms/step - loss: 0.0032 - mae: 0.0325 - val_loss: 0.0017 - val_mae: 0.0207
Epoch 6/50
200/200 ————— 2s 12ms/step - loss: 0.0033 - mae: 0.0336 - val_loss: 0.0017 - val_mae: 0.0212
Epoch 7/50
200/200 ————— 2s 12ms/step - loss: 0.0031 - mae: 0.0323 - val_loss: 0.0016 - val_mae: 0.0197
Epoch 8/50
200/200 ————— 2s 12ms/step - loss: 0.0031 - mae: 0.0319 - val_loss: 0.0019 - val_mae: 0.0246
Epoch 9/50
200/200 ————— 2s 11ms/step - loss: 0.0032 - mae: 0.0330 - val_loss: 0.0016 - val_mae: 0.0207
Test MSE: 0.006913627032190561
Test MAE: 0.057088498026132584
63/63 ————— 1s 11ms/step
MAE: 0.05708848641035098
MSE: 0.006913627095339508
RMSE: 0.0831482236451237
R2 Score: 0.783014680548945
  Actual Predicted
0  0.017685  0.017139
1  0.024707  0.016817
2  0.024967  0.024282
3  0.025228  0.024098
4  0.025748  0.026364
5  0.048114  0.028067
6  0.176593  0.049391
7  0.178414  0.173958
8  0.647074  0.143014
9  0.652016  0.616795

```

Code Example: Building a Basic RNN—Plotting the Result

```
import matplotlib.pyplot as plt

# Plot actual vs predicted values
plt.figure(figsize=(10, 5))
plt.plot(y_test[:200].flatten(), color="blue", label="Actual")
plt.plot(y_pred[:200].flatten(), color="orange", label="Predicted")
plt.title("Actual vs Predicted Global Active Power")
plt.xlabel("Time Step")
plt.ylabel("Global Active Power")
plt.legend()
plt.grid(True)
plt.show()
```



Current Subsection

- 1 Recurrent Neural Networks
 - How Basic RNN Works?
 - **Why LSTM was created?**
 - How GRU is different from LSTM

Why LSTM and GRU Are Used

Problem with basic RNNs: Basic RNNs can struggle to remember information from far back in the sequence.

- They may forget earlier information in long sequences.
- Training can suffer from vanishing gradients.
- This makes long-term patterns harder to learn.

LSTM and GRU: These are improved RNN architectures designed to control memory.

- **LSTM** uses gates to decide what to remember, forget, and output.
- **GRU** is simpler than LSTM but also uses gates to manage memory.
- Both are widely used for time-series forecasting and sequence modeling.

Key Idea: RNNs process sequences, while LSTM and GRU help the model keep useful information over longer time periods.

Vanishing Gradient Problem

Vanishing gradient happens when gradients become extremely small as they are backpropagated through many layers or time steps.

For an RNN, the gradient across time contains repeated multiplications:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}}$$

If each term is smaller than 1:

$$\left| \frac{\partial h_k}{\partial h_{k-1}} \right| < 1$$

then:

$$\prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}} \rightarrow 0$$

Meaning: early time steps receive almost no learning signal.

Introduction to LSTMs

LSTMs are a type of RNN designed to handle long-term dependencies by using a cell state and gates to control the flow of information.

- **Cell State:** Carries long-term memory through the sequence.
- **Hidden State:** Represents the current output and short-term information.
- **Gates:** Control how much information is kept, added, or output.
 - **Forget Gate:** Decides what information to discard.
 - **Input Gate:** Decides what new information to store.
 - **Output Gate:** Controls what information to output.

Benefit of LSTMs

LSTMs reduce vanishing gradient issues, allowing the model to capture long-term dependencies in sequential data.

LSTM Architecture

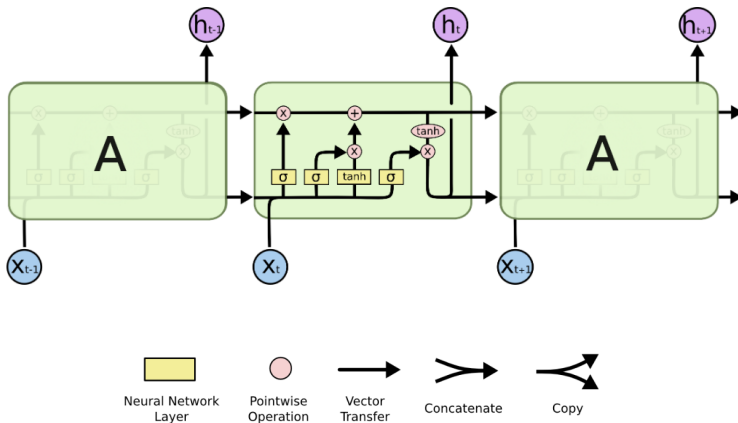


Image Ref: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

LSTM Architecture Details

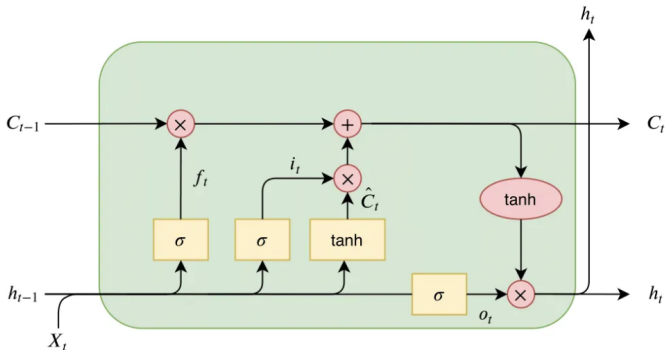


Image Ref: <https://medium.com/@saba99/long-short-term-memory-lstm-fffc5eaebfdc>

LSTM Gates and States

LSTM updates its memory using gates. Each gate has trainable weights.

$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$	Forget gate
$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$	Input gate
$\tilde{C}_t = \tanh(W_C x_t + U_C h_{t-1} + b_C)$	Candidate memory
$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	New cell state
$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$	Output gate
$h_t = o_t \odot \tanh(C_t)$	New hidden state

- x_t : current input
- h_{t-1} : previous hidden state, short-term information
- C_{t-1} : previous cell state, long-term memory
- σ : outputs values between 0 and 1, controls how much passes
- $\odot$: element-wise multiplication
- W, U, b : trainable weights and biases

How an LSTM Works: Step by Step

At time step t , the LSTM receives the current input x_t , the previous hidden state h_{t-1} , and the previous memory C_{t-1} .

1 Forget old information

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

The forget gate decides which parts of C_{t-1} should be kept or removed.

2 Choose new information

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad \tilde{C}_t = \tanh(W_C x_t + U_C h_{t-1} + b_C)$$

The input gate decides what new information is important.

3 Update the memory

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

The LSTM combines useful old memory with useful new information.

4 Produce the output

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad h_t = o_t \odot \tanh(C_t)$$

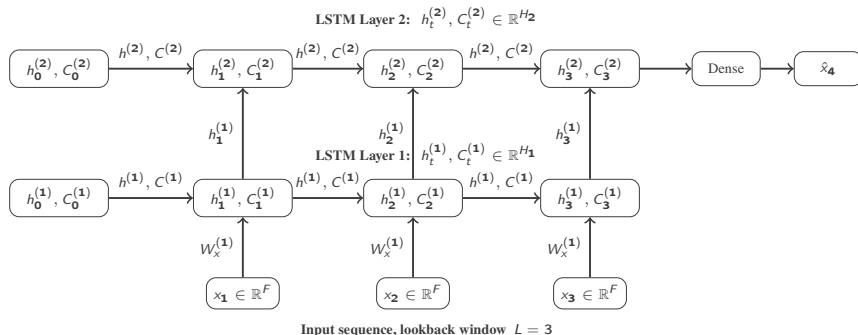
The output gate decides what part of the memory becomes the hidden state.

Information Flow Between LSTM Layers

Example: Use $L = 3$, $F = 1$, $H_1 = 8$, and $H_2 = 4$.

$$x_t \in \mathbb{R}^F, \quad h_t^{(1)}, C_t^{(1)} \in \mathbb{R}^{H_1}, \quad h_t^{(2)}, C_t^{(2)} \in \mathbb{R}^{H_2}$$

$$(h_t, C_t) = \text{LSTM}(x_t, h_{t-1}, C_{t-1})$$



Key Idea: Layer 1 reads the original input sequence x_t . Layer 2 reads the hidden-state sequence from Layer 1, $h_t^{(1)}$. The cell state C_t flows only through time inside its own LSTM layer.

Code Example—LSTM (Same Preprocessing as RNN)

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# 1. Load dataset
df = pd.read_csv("household_power_consumption.txt", sep=";", low_memory=False)

# 2. Keep target column
df = df[["Global_active_power"]]

# 3. Convert to numeric and remove missing values
df["Global_active_power"] = pd.to_numeric(df["Global_active_power"], errors="coerce")
df = df.dropna()

# 4. Use a small part for now
data = df["Global_active_power"].values[:10000].reshape(-1, 1)

# 5. Scale data
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)

# 6. Choose lookback window L
L = 24

# 7. Create input sequences and targets
X = []
y = []

for i in range(len(data_scaled) - L):
    X.append(data_scaled[i:i+L])
    y.append(data_scaled[i+L])

X = np.array(X)
y = np.array(y)
```

Code Example—LSTM (Same Preprocessing as RNN)

```
# 8. Show shapes
print("Lookback window L:", L)
print("X shape:", X.shape)
print("y shape:", y.shape)

# 9. Show one training sample
print("\nFirst input sequence:")
print(X[0].flatten())

print("\nFirst target:")
print(y[0])
```

```
Lookback window L: 24
X shape: (9976, 24, 1)
y shape: (9976, 1)
```

```
First input sequence:
[0.52301691 0.67178153 0.67360208 0.67542263 0.45149545 0.43250975
 0.45617685 0.45591678 0.45175553 0.45097529 0.55318596 0.67854356
 0.65409623 0.65981795 0.50195059 0.41482445 0.4          0.42080624
 0.39947984 0.45955787 0.74122237 0.97685306 0.88842653 0.64759428]
```

```
First target:
[0.55656697]
```

Code Example—LSTM—Building the model

```
# LSTM model

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(L, 1)),
    LSTM(32),
    Dense(1)
])

model.compile(optimizer="adam", loss="mse", metrics=["mae"])

model.summary()
```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning
super().__init__(**kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 24, 64)	16,896
lstm_1 (LSTM)	(None, 32)	12,416
dense (Dense)	(None, 1)	33

Total params: 29,345 (114.63 KB)
Trainable params: 29,345 (114.63 KB)
Non-trainable params: 0 (0.00 B)

Code Example—LSTM—Train, Test, Predict (Same as RNN)

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from tensorflow.keras.callbacks import EarlyStopping
import numpy as np
import pandas as pd

# 1. Train, test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, shuffle=False)

# 2. Early stopping
early_stop = EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True)

# 3. Train model
history = model.fit(X_train, y_train, validation_split=0.20, epochs=50, batch_size=32, callbacks=[early_stop], verbose=1)

# 4. Evaluate model
test_loss, test_mae = model.evaluate(X_test, y_test, verbose=0)

print("Test MSE:", test_loss)
print("Test MAE:", test_mae)

# 5. Predict
y_pred = model.predict(X_test)

# 6. Extra evaluation metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)

# 7. Compare actual vs predicted values
results = pd.DataFrame({
    "Actual": y_test.flatten(),
    "Predicted": y_pred.flatten()
})
print(results.head(10))

```

Code Example—LSTM—Result

```

200/200 ————— 5s 20ms/step - loss: 0.0035 - mae: 0.0319 - val_loss: 0.0017 - val_mae: 0.0210
Epoch 6/50
200/200 ————— 4s 21ms/step - loss: 0.0033 - mae: 0.0319 - val_loss: 0.0015 - val_mae: 0.0210
Epoch 7/50
200/200 ————— 6s 28ms/step - loss: 0.0032 - mae: 0.0299 - val_loss: 0.0014 - val_mae: 0.0181
Epoch 8/50
200/200 ————— 5s 23ms/step - loss: 0.0031 - mae: 0.0297 - val_loss: 0.0015 - val_mae: 0.0169
Epoch 9/50
200/200 ————— 5s 23ms/step - loss: 0.0031 - mae: 0.0289 - val_loss: 0.0014 - val_mae: 0.0163
Epoch 10/50
200/200 ————— 5s 21ms/step - loss: 0.0030 - mae: 0.0282 - val_loss: 0.0014 - val_mae: 0.0166
Epoch 11/50
200/200 ————— 5s 20ms/step - loss: 0.0030 - mae: 0.0275 - val_loss: 0.0014 - val_mae: 0.0180
Epoch 12/50
200/200 ————— 5s 25ms/step - loss: 0.0030 - mae: 0.0296 - val_loss: 0.0015 - val_mae: 0.0227
Epoch 13/50
200/200 ————— 4s 20ms/step - loss: 0.0030 - mae: 0.0288 - val_loss: 0.0014 - val_mae: 0.0179
Epoch 14/50
200/200 ————— 4s 19ms/step - loss: 0.0030 - mae: 0.0290 - val_loss: 0.0015 - val_mae: 0.0185
Epoch 15/50
200/200 ————— 5s 27ms/step - loss: 0.0030 - mae: 0.0285 - val_loss: 0.0014 - val_mae: 0.0171
Test MSE: 0.006798282731324434
Test MAE: 0.051738739013671875
63/63 ————— 2s 23ms/step
MAE: 0.05173874308607958
MSE: 0.006798282169831834
RMSE: 0.08245169597911152
R2 Score: 0.7866347999397076
  Actual Predicted
0  0.017685  0.012370
1  0.024707  0.013712
2  0.024967  0.019979
3  0.025228  0.021325
4  0.025748  0.023667
5  0.048114  0.025773
6  0.176593  0.042974
7  0.178414  0.141730
8  0.647074  0.131018
9  0.652016  0.641985

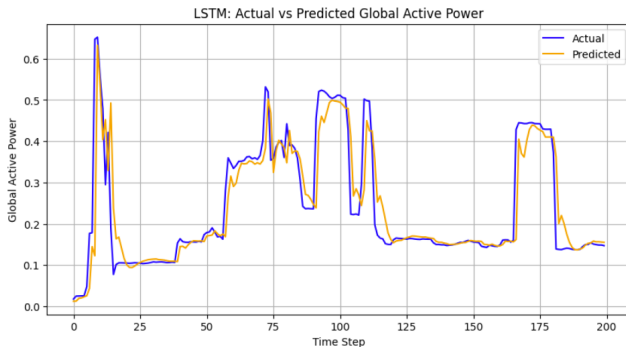
```

Code Example—Plotting Result

```
import matplotlib.pyplot as plt

# Plot actual vs predicted values
plt.figure(figsize=(10, 5))
plt.plot(y_test[:200].flatten(), color="blue", label="Actual")
plt.plot(y_pred[:200].flatten(), color="orange", label="Predicted")
plt.title("LSTM: Actual vs Predicted Global Active Power")
plt.xlabel("Time Step")
plt.ylabel("Global Active Power")
plt.legend()
plt.grid(True)
plt.show()
```

...



Current Subsection

- 1 Recurrent Neural Networks
 - How Basic RNN Works?
 - Why LSTM was created?
 - How GRU is different from LSTM

GRU: Gated Recurrent Unit

Main Idea: A GRU is a simpler version of an LSTM. It also controls memory using gates, but it has fewer gates and no separate cell state.

GRU equations:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad \text{Update gate}$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \text{Reset gate}$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad \text{Candidate hidden state}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \text{New hidden state}$$

How GRU is different from LSTM:

- GRU has **two gates**: update gate and reset gate.
- LSTM has **three gates**: forget, input, and output gates.
- GRU has only one state, h_t .
- LSTM has two states, h_t and C_t .
- GRU usually trains faster because it has fewer parameters.

Key Idea: GRU is easier and lighter than LSTM, while still helping with long-term dependencies.

All Architectures Summary

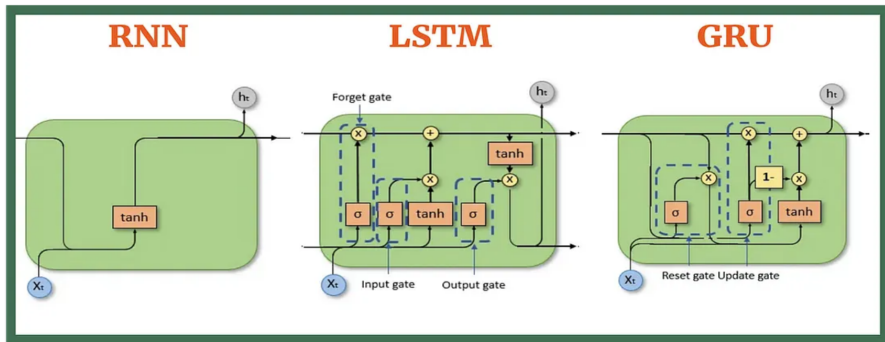


Image Ref: GRU Explained Article

GRU code

```

▶ # GRU model

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense

model = Sequential([
    GRU(64, return_sequences=True, input_shape=(L, 1)),
    GRU(32),
    Dense(1)
])

model.compile(optimizer="adam", loss="mse", metrics=["mae"])

model.summary()

... /usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning
    super().__init__(**kwargs)
Model: "sequential_1"

```

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 24, 64)	12,864
gru_1 (GRU)	(None, 32)	9,408
dense_1 (Dense)	(None, 1)	33

Total params: 22,305 (87.13 KB)
 Trainable params: 22,305 (87.13 KB)
 Non-trainable params: 0 (0.00 B)

Congratulations!

Now you know how RNNs work!

